CÔNG NGHỆ PHẦN MỀM NÂNG CAO

CHƯƠNG IV. ƯỚC LƯỢNG GIÁ PHẦN MỀM

NỘI DUNG CHÍNH

- 3.1 Giới thiệu ước lượng giá phần mềm
- 3.2 Ước lượng kích thước phần mềm
- 3.3 Ước lượng chi phí phần mềm

1. GIỚI THIỆU ƯỚC LƯỢNG GIÁ PHẦN MỀM

- Các yếu tố cần ước lượng
  - Kích thước phần mềm
  - Công sức phát triển
  - Thời gian thực hiện
- Nguyên tắc ước lượng
  - Phân rã dự án theo các chức năng chính và ước lượng theo từng chức năng
  - Dựa trên kinh nghiệm, dữ liệu quá khứ

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

- Ước lượng kích thước phần mềm
  - Qua dòng lệnh: Ước lượng trực tiếp với từng module
  - Qua điểm chức năng: Ước lượng gián tiếp thông qua số lượng input/output, yêu cầu,…

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua dòng lệnh

- Qua dòng lệnh
  - Theo đơn vị một dòng lệnh **LOC** (Lines Of Code)
  - Theo đơn vị một ngàn dòng lệnh **KDSI / KLOC (**Kilo Delivered Source Instructions/ Kilo Lines of Code)
  - Phụ thuộc ngôn ngữ lập trình

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua dòng lệnh

- *Các vấn đề gặp phải với các phương pháp LOC và KDSI*
  - Tính toán kích thước tại các giai đoạn khác nhau: phân tích yêu cầu, …
  - Cài đặt trên các ngôn ngữ lập trình khác nhau: C, Java, Lisp,…
  - Cách tính sử dụng mã lệnh: mã lệnh thực thi, định nghĩa dữ liệu,…
  - Sinh mã tự động, thiết kế giao diện trực tiếp (GUI)
  - *Giá thành của sản phẩm phụ thuộc vào ước lượng LOC*

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua điểm chức năng (**FP** - Functional Points)

- Độc lập với ngôn ngữ lập trình
- Các điểm chức năng
  - Input Item (Inp): Số input của người dùng mà chúng thực hiện các thay đổi trong cấu trúc dữ liệu (xử lý), không điều khiển sự thực hiện chương trình
  - Output Item (Oup): Số output được kết xuất cho người dùng
  - Inquiry (Inq): Số input (truy vấn) điều khiển sự thực hiện chương trình, không thay đổi trong cấu trúc dữ liệu
  - Master File (Maf) : Số tập tin được sinh ra và được sử dụng, duy trì bởi hệ thống
  - Interface (Inf): Số output đi vào ứng dụng khác hay được chia sẻ với ứng dụng nào đó

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua điểm chức năng (**FP** - Functional Points)

- Bảng giá trị các điểm chức năng theo độ phức tạp từ thấp, trung bình đến cao

	Level of Complexity		
Component	**Simple**	**Average**	**Complex**
Input item *(Inp)*	3	4	6
Output item *(Out)*	4	5	7
Inquiry *(Inq)*	3	4	6
Master file *(Maf)*	7	10	15
Interface *(inf)*	5	7	10

- Công thức tính số điểm chức năng thô

$$UFP = a_1 \times Inp + a_2 \times Oup + a_3 \times Inq + a_4 \times Maf + a_5 \times Inf$$

Với a_1, a_2, a_3, a_4, a_5 là giá trị các điểm chức năng theo độ phức tạp cho trong bảng trên.

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua điểm chức năng (**FP** - Functional Points)

Table 2. FP Counting Weights

For Internal Logical Files and External Interface Files

	Data Elements		
Record Elements	**1 - 19**	**20 - 50**	**51+**
1	Low	Low	Avg.
2 - 5	Low	Avg.	High
6+	Avg.	High	High

For External Output and External Inquiry

	Data Elements		
File Types	**1 - 5**	**6 - 19**	**20+**
0 or 1	Low	Low	Avg.
2 - 3	Low	Avg.	High
4+	Avg.	High	High

For External Input

	Data Elements		
File Types	**1 - 4**	**5 - 15**	**16+**
0 or 1	Low	Low	Avg.
2 - 3	Low	Avg.	High
3+	Avg.	High	High

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua điểm chức năng (**FP** - Functional Points)

- Công thức tính điểm chức năng FP

FP = Điểm chức năng thô x (0.65 + 0.01 x
Tổng các mức độ ảnh hưởng của các hệ số kỹ thuật)

- 14 hệ số kỹ thuật:
  - có mức độ ảnh hưởng nằm trong phạm vi
    - từ 0: không quan trọng hay không thích hợp hay không ảnh hưởng
    - tới 5: cực kỳ quan trọng hay cần thiết tuyệt đối hay ảnh hưởng nhất

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua điểm chức năng (**FP** - Functional Points)

- Các hệ số kỹ thuật

1. Data communication
2. Distributed function
3. Performance
4. Heavily used configuration
5. Transaction rate
6. Online data entry
7. End-user efficiency
8. Online update
9. Complex processing
10. Reusability
11. Installation ease
12. Operation ease
13. Multiple site
14. Facilitate change

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua điểm chức năng (**FP** - Functional Points)

- Điểm chức năng FP có thể được dùng để dự đoán số dòng lệnh LOC

LOC = AVC * số điểm chức năng FP

với AVC: yếu tố phụ thuộc vào ngôn ngữ lập trình được sử dụng

Ghi chú:

AVC = Average Value per Count (hay còn gọi là hệ số quy đổi->AVC là số dòng lệnh (LOC) trung bình cho 1 Function Point**)**

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua điểm chức năng (**FP** - Functional Points)

	Programming Statements per Function Point		
Language	**Minimum (Minus 1 Standard Deviation)**	**Mode (Most Common Value)**	**Maximum (Plus 1 Standard Deviation)**
Ada 83	45	80	125
Ada 95	30	50	70
C	60	128	170
C#	40	55	80
C++	40	55	140
Cobol	65	107	150
Fortran 90	45	80	125
Fortran 95	30	71	100
Java	40	55	80
Macro Assembly	130	213	300
Perl	10	20	30
Second generation default (Fortran 77, Cobol, Pascal, etc.)	65	107	160
Smalltalk	10	20	40
SQL	7	13	15
Third generation default (Fortran 90, Ada 83, etc.)	45	80	125
Microsoft Visual Basic	15	32	41

Source: Adapted from *Estimating Software Costs* (Jones 1998), *Software Cost Estimation with Cocomo II* (Boehm 2000), and *Estimating Software Intensive Systems* (Stutzke 2005).

2. ƯỚC LƯỢNG KÍCH THƯỚC PHẦN MỀM

Qua điểm chức năng (**FP** - Functional Points)

- Bài tập:
  - Một sản phẩm phần mềm được ước lượng có 04 đầu vào đơn giản, 9 đầu vào phức tạp, 17 đầu ra trung bình, 20 truy vấn phức tạp, 13 tập tin chính đơn giản, 15 giao diện trung bình và 7 giao diện phức tạp

1. Hãy tính số lượng điểm chức năng thô
2. Nếu hầu hết các nhân tố kỹ thuật đều không quan trọng, trừ 02 nhân tố (dễ cài đặt và dễ sử dụng) và rất quan trọng thì số lượng điểm chức năng là bao nhiêu?
3. Hãy quy đổi số lượng dòng mã lệnh theo ngôn ngữ lập trình C?

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

- Ước lượng chi phí:
  - Dựa trên kích thước, độ phức tạp
  - Dựa vào dữ liệu quá khứ
  - Chi phí tỉ lệ với công sức (effort) phát triển phần mềm
  - Chi phí tính dựa vào công sức cho các giai đoạn phát triển phần mềm: khảo sát yêu cầu, phân tích, thiết kế, cài đặt, kiểm thử, nhưng chưa tính đến giai đoạn bảo trì
  - Đơn vị tính của công sức: Người-tháng (MM: Man Month), hoặc Người-ngày (MD: Man Day)

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Các phương pháp ước lượng chi phí

Phương pháp	Giải thích
Theo ý kiến chuyên gia	Một hay nhiều chuyên gia trong cả lĩnh vực ứng dụng và phát triển phần mềm, sử dụng **kinh nghiệm** của họ để dự tính chi phí phần mềm. Quy trình này được lặp đi lặp lại cho đến khi đạt được sự nhất trí.
Theo giải thuật	Mô hình được xây dựng dựa trên các số liệu chi phí lịch sử liên quan đến một số số đo phần mềm (thường là **kích thước**) so với chi phí dự án. Ước lượng được tính dựa trên số đo đó và mô hình sẽ dự đoán chi phí cần thiết.
Bằng sự tương tự	Kỹ thuật này được áp dụng dựa trên **các dự án tương tự** đã hoàn thành trong cùng lĩnh vực. Chi phí của dự án mới sẽ được ước tính bằng cách phỏng tính từ các dự án tương tự đã hoàn thành. Myers (Myers, 1989) đã mô tả rất chi tiết phương pháp này.
Bằng luật Parkinson	Luật Parkinson nói rằng công việc sẽ được thực hiện dựa trên **khung thời gian sẵn có**. Chi phí được xác định **bởi số lượng nhân công sẵn có thay vì bởi việc đánh giá theo mục tiêu dự án**. Nếu phần mềm phải bàn giao sau 12 tháng và chỉ có 5 nhân viên, chi phí sẽ được tính là 12x5=60 người-tháng.
Đấu giá (Pricing to win)	Chi phí phần mềm được ước tính dựa trên mức độ sẵn sàng **chi tiêu của khách hàng**. Chi phí được ước tính dựa trên ngân sách của khách hàng, chứ không phải dựa trên tính năng của phần mềm.

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo ý kiến chuyên gia (Expert judgment)

- Cách thức:
  - Một hay nhiều chuyên gia trong cả lĩnh vực ứng dụng và phát triển phần mềm, sử dụng kinh nghiệm của họ để dự tính chi phí phần mềm.
  - Quy trình này được lặp đi lặp lại cho đến khi đạt được sự nhất trí.
- Thuận lợi:
  - Phương pháp dự đoán chi phí thấp một cách tương đối
  - Có thể chính xác nếu các chuyên gia có kinh nghiệm trực tiếp trong các hệ thông tương tự
- Khó khăn:
  - Rất thiếu chính xác nếu không có các chuyên gia thực sự!

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật

- Một số mô hình ước lượng chi phí theo giải thuật
  - Mô hình Walston và Felix
  - Mô hình Bailey và Basili
  - Mô hình COCOMO

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình Walston và Felix

- Một trong các mô hình giải thuật sớm nhất (1977)
- Mô hình được đề nghị sau khi nghiên cứu 60 dự án của IBM
- Có 29 yếu tố ảnh hưởng tới hiệu suất
- Công thức ước lượng *công sức E*

$$E = 5.2 \times S^{0.91} \text{ (người - tháng)}$$

Với S là kích thước được ước lượng của hệ thống *(theo KDSI)*

- Công thức ước lượng *thời gian thực hiện T*

$$T = 2.5 \times E^{0.35} \text{ (tháng)}$$

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình Walston và Felix

- Mỗi yếu tố sẽ nhận 1 trong 3 giá trị tùy thuộc vào sự tác động của nó tới hiệu suất
  - 1 (cao): làm tăng hiệu suất
  - 0 (trung bình): không ảnh hưởng tới hiệu suất
  - -1 (thấp): làm giảm hiệu suất

Mô hình Walston và Felix

1. Customer interface complexity	16. Use of design and code inspections
2. User participation in requirements definition	17. Use of top-down development
3. Customer-originated program design changes	18. Use of a chief programmer team
4. Customer experience with the application area	19. Overall complexity of code
5. Overall personnel experience	20. Complexity of application processing
6. Percentage of development programmers who participated in the design of functional specifications	21. Complexity of program flow
7. Previous experience with the operational computer	22. Overall constraints on program's design
8. Previous experience with the programming language	23. Design constraints on the program's main storage
9. Previous experience with applications of similar size and complexity	24. Design constraints on the program's timing
10. Ratio of average staff size to project duration (people per month)	25. Code for real-time or interactive operation or for execution under severe time constraints
11. Hardware under concurrent development	26. Percentage of code for delivery
12. Access to development computer open under special request	27. Code classified as nonmathematical application and input/output formatting programs
13. Access to development computer closed	28. Number of classes of items in the database per 1000 lines of code
14. Classified security environment for computer and at least 25% of programs and data	29. Number of pages of delivered documentation per 1000 lines of code
15. Use of structured programming	

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình Bailey và Basili

- Mô hình được đề xuất năm 1981 bởi Bailey và Basili
- Mô hình này sử dụng CSDL của 18 dự án viết bằng ngôn ngữ Fortran tại trung tâm Goddard Space Flight của NASA
- Các nhóm yếu tố ảnh hưởng tới công sức: phương pháp, độ phức tạp và kinh nghiệm
- Công thức ước lượng công sức ban đầu E

$$E = 5.5 + 0.73 \text{ x } S^{1.16} \text{ (người - tháng)}$$

Với S là kích thước được ước lượng của hệ thống *(theo KDSI)*

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình Bailey và Basili

- Mỗi yếu tố ảnh hưởng tới công sức nhận một trong các giá trị từ 0 đến 5

Total methodology (METH)	*Cumulative complexity (CPLX)*	*Cumulative experience (EXP)*
Tree charts	Customer interface complexity	Programmer qualifications
Top-down design	Application complexity	Programmer machine experience
Formal documentation	Program flow complexity	Programmer language experience
Chief programmer teams	Internal communication complexity	Programmer application experience
Formal training	Database complexity	Team experience
Formal test plans	External communication complexity	
Design formalisms	Customer-initiated program design changes	
Code reading		
Unit development folders		

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 81 cơ bản

- Được đề nghị bởi Boehm
  - Dạng cơ bản (organic): áp dụng cho nhóm nhỏ, môi trường quen thuộc
  - Dạng trung bình (semi-detached): áp dụng cho dự án khá lớn, có một ít kinh nghiệm
  - Dạng lớn (embeded): áp dụng cho dự án lớn, môi trường mới
- Bảng mức độ khó khi phát triển sản phẩm

Software project	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 81 cơ bản

- Công sức:

$$E = a_b \times S^{b_b}$$

- Thời gian:

$$T = c_b \times E^{d_b}$$

- Số người:

$$P = E / T$$

  - a_b, b_b, c_b, d_b: được xác định dựa vào bảng mức độ khó khi phát triển phần mềm
  - S: kích thước được ước lượng của hệ thống (*theo KDSI)*

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 81 trung gian

- Công sức: $E = a_b \times S^{b_b} \times EAF$
- Thời gian: $T = c_b \times E^{d_b}$
- Số người: $P = E / T$
  - a_b và b_b: xác định dựa vào bảng mức độ khó khi phát triển phần mềm dưới đây
  - c_b và d_b: xác định theo như bảng của COCOMO 81 cơ bản
  - EAF (effort adjustment factor): hệ số hiệu chỉnh công sức. Nó được tính bằng tích các hệ số phát triển
  - S: kích thước được ước lượng của hệ thống (*theo KDSI)*

Software project	a_i	b_i
Organic	3.2	1.05
Semi-detached	3.0	1.12
Embedded	2.8	1.20

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 81 trung gian

- Các hệ số phát triển

Cost Drivers	Ratings					
	Very Low	Low	Nominal	High	Very High	Extra High
Product attributes						
Required software reliability	0.75	0.88	1.00	1.15	1.40	
Size of application database		0.94	1.00	1.08	1.16	
Complexity of the product	0.70	0.85	1.00	1.15	1.30	1.65
Hardware attributes						
Run-time performance constraints			1.00	1.11	1.30	1.66
Memory constraints			1.00	1.06	1.21	1.56
Volatility of the virtual machine environment		0.87	1.00	1.15	1.30	
Required turnabout time		0.87	1.00	1.07	1.15	
Personnel attributes						
Analyst capability	1.46	1.19	1.00	0.86	0.71	
Applications experience	1.29	1.13	1.00	0.91	0.82	
Software engineer capability	1.42	1.17	1.00	0.86	0.70	
Virtual machine experience	1.21	1.10	1.00	0.90		
Programming language experience	1.14	1.07	1.00	0.95		
Project attributes						
Application of software engineering methods	1.24	1.10	1.00	0.91	0.82	
Use of software tools	1.24	1.10	1.00	0.91	0.83	
Required development schedule	1.23	1.08	1.00	1.04	1.10	

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 81 trung gian

- Bài tập:
  - Bạn được giao phát triển một phần mềm dạng lớn có 128000 LOC.
  - Các thông số nhìn chung đều ảnh hưởng ở mức bình thường, nhưng:
    - yêu cầu về độ tin cậy của phần mềm là rất cao,
    - sản phẩm có tính chất phức tạp,
    - ràng buộc về lưu trữ chính là vô cùng cao,
    - yêu cầu chặt chẽ về lịch biểu phát triển,
  - Ngoài ra đội ngũ lập trình viên được đánh giá có kinh nghiệm và khả năng tốt.
  - Sử dụng mô hình COCOMO 81 trung gian để tính E, T và P?

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình COCOMO 81 được phát triển trên giả thiết sử dụng tiến trình phát triển phần mềm theo mô hình thác nước, và tất cả các phần mềm đều được phát triển từ đầu.
- Mô hình COCOMO 2 được thiết kế để thích ứng với những cách tiếp cận khác nhau đối với sự phát triển phần mềm.

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình COCOMO 2 kết hợp chặt chẽ các mô hình con nhằm đưa ra các dự đoán phần mềm chi tiết
- Các mô hình con trong COCOMO 2:
  - Mô hình Application Composition: mô hình này giả định rằng hệ thống được tạo nên từ các thành phần **có thể tái sử dụng**. Mô hình này được thiết kế để ước lượng công sức phát triển bản mẫu.
  - Mô hình Early design: được sử dụng tại giai đoạn thiết kế kiến trúc khi **các yêu cầu đã có sẵn** (và thiết kế chi tiết vẫn chưa bắt đầu).

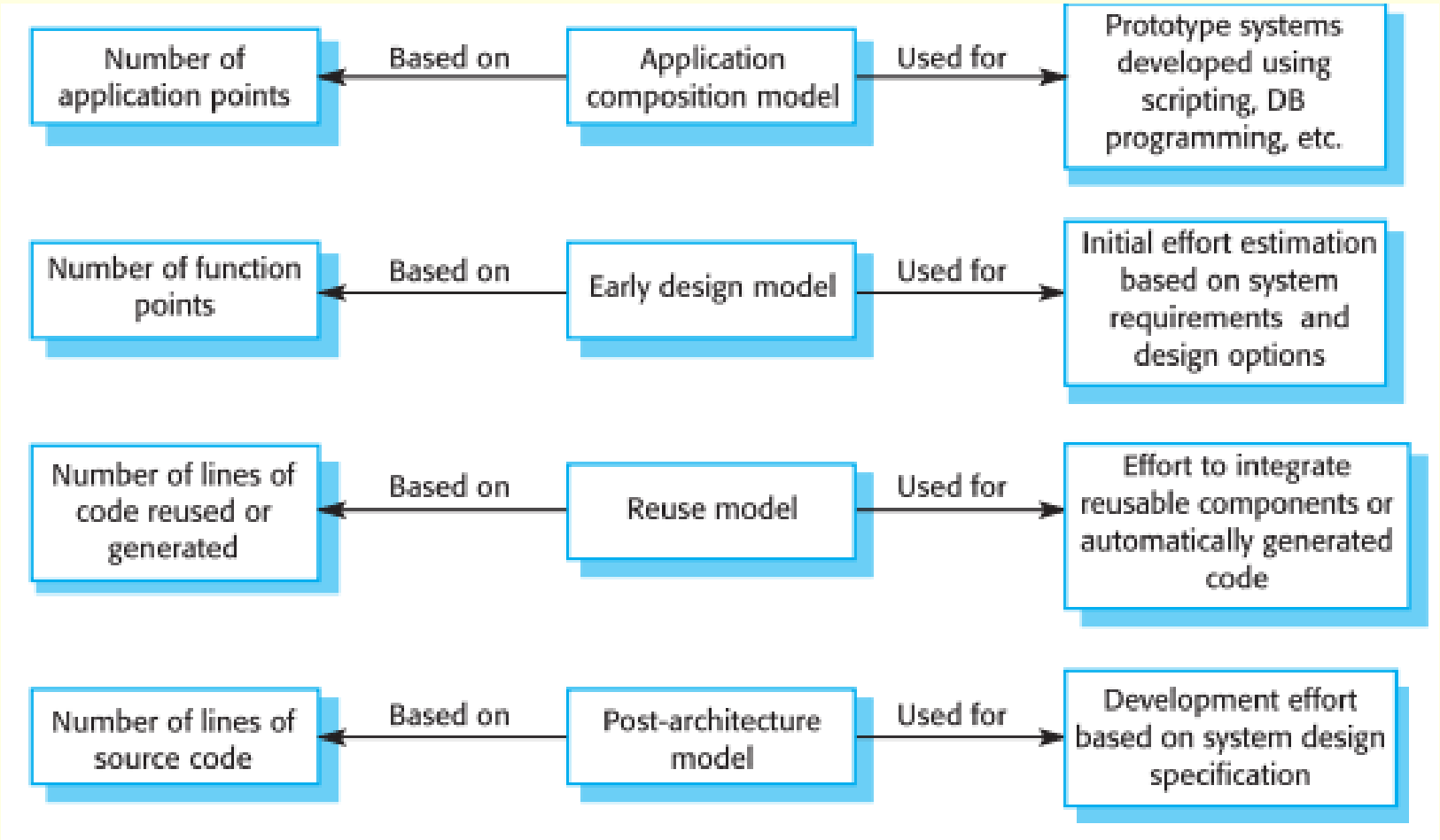
3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Các mô hình con trong COCOMO 2:
  - Mô hình Reuse: được sử dụng để tính công sức tích hợp các thành phần có thể sử dụng lại và/hoặc mã chương trình được tự động sinh ra bởi các công cụ dịch hay thiết kế chương trình. Nó thường được sử dụng kết hợp với mô hình Post-architecture.
  - Mô hình Post-architecture: được sử dụng ngay khi kiến trúc hệ thống đã được thiết kế và các thông tin chi tiết hơn về hệ thống đã có sẵn.

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2



3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Application Composition
  - Dùng để ước lượng công sức lập bản mẫu dự án và áp dụng cho các dự án được phát triển bằng cách kết hợp các thành phần sẵn có.
  - Ước lượng dựa trên số lượng điểm ứng dụng (Application Point)
  - Công thức ước lượng công sức

$$E = (NAP \times (1 - \%reuse/100)) / PROD \text{ (người — tháng)}$$

    - NAP: số lượng điểm ứng dụng
    - %reuse: % mã lệnh được tái sử dụng từ các dự án khác
    - PROD: hiệu suất. Nó phụ thuộc vào kinh nghiệm và khả năng của nhà phát triển, cũng như tính trưởng thành và khả năng của công cụ.

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Application Composition
  - Bảng xác định hiệu suất PROD

Developer's experience and capability	Very low	L w	Nominal	High	Very high
CASE maturity and capability	Very low	Low	Nominal	High	Very high
PROD (NOP/month)	4	7	13	25	50

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Application Composition
  - Số lượng điểm ứng dụng NAP phụ thuộc vào:
    - Các màn hình hiển thị riêng biệt (giao diện người dùng)
    - Các báo cáo được sinh ra bởi hệ thống
    - Các thành phần thư viện

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Application Composition
  - Tính số lượng điểm ứng dụng
    - Đếm số lượng màn hình, báo cáo và module
    - Xác định độ phức tạp cho từng thành phần (màn hình / báo cáo / module) theo bảng sau:

For Screens ành ph				*For Reports* ành ph			
	Number and source of data tables				*Number and source of data tables*		
Number of views contained	Total < 4 (<2 server, <3 client)	Total < 8 (2-3 server, 3-5 client)	Total 8+ (>3 server, >5 client)	*Number of sections contained*	Total < 4 (<2 server, <3 client)	Total < 8 (2-3 server, 3-5 client)	Total 8+ (>3 server, >5 client)
<3	simple	simple	medium	0 or 1	simple	simple	medium
3 - 7	simple	medium	difficult	2 or 3	simple	medium	difficult
8 +	medium	difficult	difficult	4 +	medium	difficult	difficult

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Application Composition
  - Tính số lượng điểm ứng dụng
    - Tính số điểm ứng dụng cho từng thành phần (màn hình / báo cáo / module) sau khi biết độ phức tạp theo bảng sau:

Object type	*Simple*	*Medium*	*Difficult*
Screen	1	2	3
Report	2	5	8
3GL component	-	-	10

    - Tính tổng số điểm ứng dụng cho tất cả các thành phần

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Early Design
  - Ước lượng công sức khi các yêu cầu đã được chấp nhận và thiết kế hệ thống đang được thực hiện
  - Công thức ước lượng công sức:

$$E = a \times S^b \times M$$

    - M: tích của 7 hệ số nhân
    - a= 2.94
    - S: kích thước được ước lượng của hệ thống (theo KDSI)
    - b: thay đổi trong khoảng 1.1 đến 1.24 tùy theo tính mới của hệ thống, tính linh động trong phát triển, các phương pháp quản lý rủi ro và tính trưởng thành của tiến trình.

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Early Design
  - Các hệ số nhân phản ánh khả năng của nhà phát triển, các yêu cầu phi chức năng, sự hiểu biết rõ về nên tảng phát triển, …
    - RCPX: product reliability and complexity
    - RUSE: the reuse required
    - PDIF: platform difficulty
    - PREX: personal experience
    - PERS: personal capability
    - SCED: required schedule
    - FCIL: the team support facilities

  Giá trị của các hệ số này nằm trong khoảng từ 1 (rất thấp) đến 6 (vô cùng cao)

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Reuse
  - Ước lượng công sức tích hợp mã lệnh được sinh ra hay có thể tái sử dụng
  - COCOMO 2 xem mã lệnh được tái sử dụng ở 1 trong 2 dạng:
    - Mã tái sử dụng hộp đen: mã lệnh được tái sử dụng mà không cần hiểu hay sửa đổi nó ➔ công sức phát triển cho mã hộp đen được tính là 0
    - Mã tái sử dụng hộp trắng: mã lệnh được chỉnh sửa để tích hợp với mã lệnh mới hay các thành phần được tái sử dụng khác ➔ cần tính đến công sức phát triển

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Reuse
  - Nhiều hệ thống còn bao gồm các mã lệnh được tự động sinh ra bởi các bộ dịch chương trình (một dạng tái sử dụng)
  - Đối với mã lệnh sinh tự động, dự đoán công sức để tích hợp mã lệnh này:

$$E = (ASLOC * AT/100)/ATPROD$$

    - ASLOC: số dòng mã lệnh trong các thành phần được chỉnh sửa (tái sử dụng hộp trắng)
    - AT: tỷ lệ % mã lệnh sinh tự động được chỉnh sửa
    - ATPROD: hiệu suất của các kỹ sư trong việc tích hợp mã lệnh đó (khoảng 2400 câu lệnh nguồn / tháng)

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Reuse
  - Dự đoán số dòng mã nguồn mới tương đương ESLOC với số mã lệnh được tái sử dụng:
    - ESLOC: tính đến công sức hiểu phần mềm, tạo ra các thay đổi đối với mã lệnh được tái sử dụng, tạo ra các thay đổi đối với hệ thống để tích hợp mã lệnh đó. Nó còn tính đến số mã lệnh sinh tự động.

$$ESLOC = ASLOC * (1-AT/100) * AAM$$

    - AT, ATPROD: như trước
    - AAM: hệ số nhân hiệu chỉnh thích ứng. Được tính từ chi phí sửa mã lệnh được tái sử dụng, các chi phí để hiểu cách tích hợp mã lệnh và chi phí ra quyết định tái sử dụng.

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Post-architecture
  - Sử dụng cùng công thức như mô hình Early Design nhưng có tới 17 hệ số nhân
  - Công thức ước lượng công sức:

$$E = a \times S^b \times M$$

    - M: tích của 17 hệ số nhân
    - a= 2.94
    - S: kích thước được ước lượng của hệ thống (theo KDSI)
    - b = 1.01 +0.01 x ΣW_i

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Post-architecture
  - 17 hệ số nhân M

Cost drivers	Rating					
	Very low	Low	Nominal	High	Very high	Extra high
Product factors						
Reliability required	0.75	0.88	1.00	1.15	1.39	
Database size		0.93	1.00	1.09	1.19	
Product complexity	0.75	0.88	1.00	1.15	1.30	1.66
Required reusability		0.91	1.00	1.14	1.29	1.49
Documentation needs	0.89	0.95	1.00	1.06	1.13	
Platform factors						
Execution time constraints			1.00	1.11	1.31	1.67
Main storage constraints			1.00	1.06	1.21	1.57
Platform volatility		0.87	1.00	1.15	1.30	
Personnel factors						
Analyst capability	1.50	1.22	1.00	0.83	0.67	
Programmer capability	1.37	1.16	1.00	0.87	0.74	
Application experience	1.22	1.10	1.00	0.89	0.81	
Platform experience	1.24	1.10	1.00	0.92	0.84	
Language and tool experience	1.25	1.12	1.00	0.88	0.81	
Personnel continuity	1.24	1.10	1.00	0.92	0.84	
Project factors						
Use of software tools	1.24	1.12	1.00	0.86	0.72	
Multi-site development	1.25	1.10	1.00	0.92	0.84	0.78
Required development schedule	1.29	1.10	1.00	1.00	1.00	

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Post-architecture
  - Kích thước mã lệnh S được tính bằng cách cộng 3 ước lượng dưới đây:
    - Ước lượng số dòng mã lệnh mới được phát triển
    - Ước lượng số dòng mã lệnh tương đương được tính từ mô hình reuse
    - Ước lượng số dòng mã lệnh phải sửa đổi theo các yêu cầu thay đổi

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Post-architecture
  - Các hệ số W

Precedentedness	Reflects the previous experience of the organisation with this type of project. Very low means no previous experience; Extra high means that the organisation is completely familiar with this application domain.
Development flexibility	Reflects the degree of flexibility in the development process. Very low means a prescribed process is used; Extra high means that the client sets only general goals.
Architecture/risk resolution	Reflects the extent of risk analysis carried out. Very low means little analysis; Extra high means a complete and thorough risk analysis.
Team cohesion	Reflects how well the development team know each other and work together. Very low means very difficult interactions; Extra high means an integrated and effective team with no communication problems.
Process maturity	Reflects the process maturity of the organisation. The computation of this value depends on the CMM Maturity Questionnaire, but an estimate can be achieved by subtracting the CMM process maturity level from 5.

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Mô hình Post-architecture
  - Các hệ số W

W(i)	Very Low	Low	Nominal	High	Very High	Extra High
Precedentedness	4.05	3.24	2.43	1.62	0.81	0.00
Development Flexibility	6.07	4.86	3.64	2.43	1.21	0.00
Architecture / Risk Resolution	4.22	3.38	2.53	1.69	0.84	0.00
Team Cohesion	4.94	3.95	2.97	1.98	0.99	0.00
Process Maturity	4.54	3.64	2.73	1.82	0.91	0.00

3. ƯỚC LƯỢNG CHI PHÍ PHẦN MỀM

Theo giải thuật: Mô hình COCOMO 2

- Bài tập:
  - Ước lượng công sức thực hiện phần mềm có khoảng 100 KDSI bằng mô hình COCOMO 2.
  - Đây là dạng phần mềm mới đối với công ty bạn.
  - Để thực hiện, công ty thành lập các nhóm. Nhóm của bạn thực hiện việc ước lượng khi kiến trúc hệ thống đã được thiết kế và các thông tin chi tiết hơn về hệ thống đã có sẵn.
  - Nhìn chung, các hệ số W và hệ số nhân ở mức độ bình thường. Tuy nhiên, công ty đặc biệt yêu cầu rất cao về sự cần thiết của tài liệu, và kinh nghiệm sử dụng công cụ và ngôn ngữ.